FlowCabal 架构设计文档

日期：2026.04.04

AI 辅助长篇写作的 DAG 工作流引擎—“文本版 ComfyUI”。

目录

FlowCabal 架构设计文档 1
问题与动机 2
  长篇小说写作的核心困难 2
  设计哲学 2
  设计演化 2
    v6：State 中心化 2
    当前设计：简化为原子操作 2
整体架构 3
  单进程 TypeScript 3
  Monorepo 结构 3
核心概念 4
  Workspace 4
  TextBlock 4
  NodeDef 4
  LLM 配置 5
执行引擎 5
  Target Set 和 Stale Roots 5
    Target Set（持久化，自动管理） 5
    Stale Roots（持久化，惰性推导） 5
    NodeStatus 计算 5
  Todo List（实时计算） 5
  执行流程 6
    简单函数模型 6
记忆架构 6
  设计哲学 6
    约束查询是核心能力 6
    Memory 是 manuscripts 的有损缓存 6
    不用 RAG 6
  种子文件 6
    生成性事实 vs 派生断言 7
  三级上下文加载 7
  Agent 工具集 7
目录结构 7
  全局配置 ~/.config/flowcabal/ 7
  项目本地 <project>/.flowcabal/ 8
LLM 集成 8
  Provider 工厂 8
  生成接口 8
架构决策记录 8
  为什么不用 RAG 8

为什么纯文件系统 9
为什么 Vercel AI SDK 9
为什么无状态原子操作 9
为什么不需要版本缓存 9
CLI 命令 9
init 9
workspace 10
llm 10
node 10
run 11
memory 11

问题与动机

长篇小说写作的核心困难

长篇小说（10 万字+）的 AI 辅助写作面临三个根本性挑战：

1. **上下文窗口不够装**：即使 200K token 的窗口也无法一次性容纳一部完整小说加全部设定。必须有选择地加载上下文
2. **一致性随长度崩溃**：角色性格漂移、设定自相矛盾、伏笔遗忘—这些不是幻觉，是**上下文缺失**的直接后果
3. **生成流程不可控**：端到端的“给我写一章”无法让作者介入。作者需要拆解、调试、迭代每一步

FlowCabal 的核心主张：**人类定义 what（工作流结构、创作意图），AI 负责 how（上下文检索、约束满足、文本生成）。**

设计哲学

“如果用户不重视生成的东西，那我们也认为它没有价值。”

- 不维护版本缓存：每次执行是独立的，用户自行决定是否保留结果
- 不自动级联失效：通过 Target Set 和 Stale Roots 驱动，用户主动参与
- CLI-first：Beta 版 99% CLI / 1% UI。每个操作都是原子命令，TUI/GUI 只是调用同样的原子操作的视觉包装。外部 Agent（Claude Code、Cursor、Cline）可以零成本通过 shell 驱动 FlowCabal

设计演化

v6：State 中心化

v6 引入了 state.ts 作为唯一内存模型，workspace 和 run 都读写它。但随着演进发现了两个根本矛盾：

1. **Workspace 无状态**：每次查询走磁盘。10 节点的 getDashboard = 20+ 次 readFile
2. **Run 私有状态**：run.ts 内部的 `outputs: Map` 对外不可见，Agent 执行期间的 query_runtime 完全废掉

当前设计：简化为原子操作

移除 State 中心化，改为**每次调用独立完成：加载→执行→持久化→返回**的无状态原子操作函数。移除 RunHandle、事件总线、auto/step 模式。

整体架构

单进程 TypeScript

```
┌──────────────────────────────────────────────────────────┐
│  Bun 进程                                                 │
│                                                           │
│  ┌──────────┐   ┌──────────┐   ┌──────────────┐          │
│  │ CLI      │→ │  Engine  │→ │  LLM Provider │  │
│  │ (yargs) │  │  (DAG +  │  │  (Vercel AI  │  │
│  │          │  │   Agent) │  │   SDK)       │  │
│  └──────────┘   └──────────┘   └──────────────┘          │
│                      │                                    │
│           ┌──────────┴──────────┐                         │
│           │                     │                         │
│  ┌────────▼────────┐   ┌──────────────▼──────────────┐   │
│  │ ~/.config/      │   │ <project>/.flowcabal/       │   │
│  │ flowcabal/      │   │ memory/ + runner-cache/     │   │
│  └─────────────────┘   └─────────────────────────────┘   │
└──────────────────────────────────────────────────────────┘
```

没有数据库、没有 WebSocket、没有子进程。一个 Bun 进程，读写文件系统。存储分两层：全局配置 + 项目本地状态。

Monorepo 结构

```
flowcabal/
  package.json              # Bun workspace root
  packages/
    engine/                 # 核心无头引擎（零 UI 依赖）
      src/
        types.ts            # 领域类型
        schema.ts           # Zod schemas
        paths.ts            # 路径注册表
        llm/                # LLM 集成
          provider.ts       # Vercel AI SDK provider 工厂
          generate.ts      # generateText 封装
        workspace/          # Workspace 子系统
          init.ts          # 从空/Workflow 初始化 Workspace
          export.ts        # 导出为 Workflow
          core/
            index.ts       # 统一导出
            node.ts        # Node 和 Block CRUD
            graph.ts       # DAG 拓扑 + 依赖 + Todo/Stale 计算
            runner.ts      # 单节点/全量执行
        agent/              # Agent 子系统
          memory-agent.ts   # Agent 执行（单次 + 流式）
          assembler.ts      # L0 index.md 加载
          memory.ts        # Memory CRUD + index 生成
          tools-memory.ts  # Agent tools 定义
          prompts.ts       # 系统提示词
    cli/                    # CLI（TUI 将来）
      src/
        index.ts            # bin: flowcabal
        commands/
          init.ts           # 项目初始化
```

```
        workspace.ts      # Workspace 管理
        llm.ts            # LLM 配置管理
        node.ts           # 节点编排
        run.ts            # 执行
        memory.ts         # 记忆管理
```

Engine 包零 UI 依赖—不 console.log、不假设 terminal。可以被任何前端（TUI、Web、Electron）使用。

核心概念

Workspace

Workspace 是项目的运行时实例，绑定一个 DAG 结构和执行状态。

```
interface Workspace {
  id: string;
  name: string;
  nodes: NodeDef[];
  outputs: Map<string, string>;       // nodeId → output
  upstream: Map<string, string[]>;     // nodeId → 依赖的节点
  downstream: Map<string, string[]>;   // nodeId → 依赖它的节点
  target_nodes: string[];              // 待执行的节点
  stale_nodes: string[];               // 可能过时的节点（提醒用）
}
```

TextBlock

节点 prompt 的构成单元，三种类型：

```
type TextBlock =
  | { kind: "literal"; content: string }
  | { kind: "ref"; nodeId: string }
  | { kind: "agent-inject"; hint: string };
```

- `literal` — 静态文本，用户直接编写
- `ref` — 引用上游节点输出，执行时动态替换为上游的 output
- `agent-inject` — Agent 注入点。`hint` 告诉 Agent 方向，Agent 读记忆自主决定注入什么内容

`ref` 同时隐式定义了 DAG 的连接关系。依赖关系通过扫描 systemPrompt + userPrompt 中的 ref 块自动推导，无需显式 Edge 类型。

NodeDef

```
interface NodeDef {
  id: string;
  label: string;
  systemPrompt: TextBlock[];
  userPrompt: TextBlock[];
}
```

节点不存储 LLM 配置。LLM 选择在运行时由以下优先级决定：

1. 命令行指定的配置
2. `~/.config/flowcabal/llm-configs.json` 中的 `"default"` 配置

LLM 配置

```
type LlmProvider =
  | "openai" | "anthropic" | "google"
  | "mistral" | "xai" | "cohere"
  | "openai-compatible";

interface LlmConfig {
  provider: LlmProvider;
  baseURL?: string;
  apiKey: string;
  model: string;
  temperature?: number;
  maxTokens?: number;
  topP?: number;
  frequencyPenalty?: number;
  presencePenalty?: number;
  providerOptions?: Record<string, Record<string, JsonValue>>;
}
```

执行引擎

Target Set 和 Stale Roots

Target Set（持久化，自动管理）

- 新建节点 → 自动加入 target_nodes
- 修改节点（addBlock/removeBlock/updateBlock）→ 自动加入 target_nodes
- 执行完毕 → 自动移出 target_nodes
- 用户可手动增删

Stale Roots（持久化，惰性推导）

- 上游节点被修改/重跑时 → 直接下游加入 stale_nodes
- 自身重跑 → 从 stale_nodes 移除，并向下游传播
- 完整的 Possibly Stale 集合按需 BFS 推导（calcStale），不实时维护

NodeStatus 计算

```
type NodeStatus = "done" | "stale" | "pending";
// done = 有 output 且不在 stale_nodes 中
// stale = 有 output 且在 stale_nodes 中（提醒用户可能过时）
// pending = 无 output
```

Todo List（实时计算）

todoList(ws) = Target Set + DAG 依赖 → Kahn 拓扑排序 → 分层级列表。纯计算，不存储。

```
interface TodoLevel {
  index: number;
  nodeIds: string[];
}
```

执行时从 targets 开始，按依赖顺序分层。用户可以指定 target 节点，软件自动解析依赖的上游节点。

执行流程

```
flowcabal run
  → calcStale(ws)           // 重新计算 stale
  → todoList(ws)            // 计算执行队列
  → for nodeId in todoList  // 按拓扑序逐个执行
      → runNode(ws, nodeId)
        → resolvePrompt     // 解析 literal / ref / agent-inject
        → generate          // 调用 LLM
        → ws.outputs.set   // 保存输出
        → ws.stale_nodes 移出 + 向下游传播
        → ws.target_nodes 移出
```

简单函数模型

```
runSingle(ws, config, rootDir) → nodeId | null
runAll(ws, config, rootDir) → nodeId[]
```

执行引擎不维护 RunHandle、事件总线、auto/step 模式。就是简单函数：给一组节点，并行或逐个执行，返回结果。编排逻辑留给调用方。

记忆架构

设计哲学

约束查询是核心能力

记忆模块的底层能力是**约束查询**：给出新内容，判断它与已有章节有没有冲突、冲突在哪里。

关键洞察：**约束查询和上下文注入是同一个能力的两面。** Agent 拿节点 prompt + 上游输出作为锚点去记忆做约束查询，查到的相关约束就是该注入的上下文。不存在独立的“检索阶段”和“生成阶段”。

Memory 是 manuscripts 的有损缓存

Memory 文件不是独立的知识库，而是 manuscripts（定稿章节）的**有损缓存**。类比 coding agent：代码库是 memory，grep 是检索。对于 FlowCabal：`manuscripts/` 是完整信息源，memory 文件是为了避免每次都加载全部原文的缓存层。

不用 RAG

RAG 的 embedding 召回基于语义相似性，但小说中语义相似的段落可能有几十上百种，召回无法区分哪个是当前需要的。Memory 的跳转链接是**因果关系驱动**的检索，比语义相似性更准确。

种子文件

init 时只创建有**真实约束力**的文件—即 Agent 无法从 manuscripts 自动衍生的东西：

文件/目录	约束域	内容
`index.md`	导航	L0 索引，Agent 的导航入口。自动生成，列出所有 memory 文件及其一行摘要
`voice.md`	叙事一致性	POV、文体、句法模式。需要句法级正例和反例，不能只是抽象标签。含类型叙事约束

characters/	角色一致性	一角色一文件。生成性事实：背景因果→性格→动机→关系。写因果链，不写特征列表
world/	设定一致性	一概念一文件。世界硬规则、体系原理、边界。含类型设定约束（如“硬科幻：无魔法”）
manuscripts/	全精度信息源	定稿章节。L2 层，通过跳转链接按需可达

生成性事实 vs 派生断言

Memory 文件应写**生成性事实**（intensional），不写**派生断言**（extensional）：

- 好：`"奥托对卡莲之死的愧疚驱动他所有决策"` — 一条生成规则，可以推导出无限多具体行为
- 坏：`"奥托不会放弃复活卡莲 / 奥托不会信任陌生人 / 奥托说话彬彬有礼"` — 试图穷举无限集合

三级上下文加载

级别	内容	何时加载
L0	index.md — 导航入口	Agent 每次启动时自动加载（assembler.ts 的 loadL0）
L1	各记忆文件（characters/ 等）	Agent 读 L0 后通过 read_file 按需加载
L2	manuscripts/ 原文	通过跳转链接按需可达

文件间通过**稀疏跳转链接**（→ path/to/file.md）构成有向图，Agent 按需导航。

Agent 工具集

工具	说明
list_memory	列出所有 memory 文件（不含 manuscripts/ 和 index.md）
list_manuscripts	列出所有手稿文件（manuscripts/ 目录下的 .md 文件）
read_file	读取指定 memory 文件，返回 { path, content }
read_manuscript	读取指定手稿文件，返回 { path, content }
write_file	全量覆写指定文件（排除 manuscripts/ 和 index.md）
delete_file	删除指定文件（排除 manuscripts/ 和 index.md）
update_index	重新生成 index.md（遍历所有 .md 文件，提取首行）

目录结构

全局配置 ~/.config/flowcabal/

```
~/.config/flowcabal/
└── llm-configs.json                # LLM 配置池（Record<name, LlmConfig>，一套
"default"）
```

项目本地 <project>/.flowcabal/

```
<project>/.flowcabal/
├── <workspace-id>/                # workspace 目录
│   └── workspace.json              # Workspace 数据
├── current/                       # 当前 workspace
│   └── workspace.json
└── memory/                         # Agent 记忆
    ├── index.md                   # L0 导航（自动生成）
    ├── voice.md                   # 文体约束
    ├── characters/                # 角色
    ├── world/                    # 世界观
    └── manuscripts/              # 定稿章节（L2）
```

路径定义见 `packages/engine/src/paths.ts`。

LLM 集成

Provider 工厂

`provider.ts` 根据 LlmConfig.provider 选择 Vercel AI SDK 的对应 provider：

provider 值	SDK 包
openai	@ai-sdk/openai
openai-compatible	@ai-sdk/openai（覆盖 baseURL）
anthropic	@ai-sdk/anthropic
google	@ai-sdk/google
mistral	@ai-sdk/mistral
xai	@ai-sdk/xai
cohere	@ai-sdk/cohere

`openai-compatible` 复用 `@ai-sdk/openai`，只是覆盖 baseURL。DeepSeek 等国产模型走这个路径。

生成接口

- `generate(config, system, prompt, abortSignal?)` — 返回完整文本。用于 Agent 的 tool calling 循环

架构决策记录

为什么不用 RAG

问题	分析
语义歧义	小说中语义相似的段落可能有几十上百种，embedding 召回无法区分哪个是当前需要的

因果盲区	伏笔和回收在语义空间里往往很远，余弦相似度捕捉不到因果链
事实幻觉	召回错误段落导致的不是普通幻觉，而是事实性幻觉—基于错误上下文的逻辑自洽输出
基础设施	不需要 embedding 模型、向量数据库、索引构建

Agent 驱动的导航（L0 索引 + 跳转链接）比语义相似性更准确，因为链接是因果关系驱动的。

为什么纯文件系统

维度	分析
可读性	Markdown 文件可直接阅读、编辑—人和 Agent 共同读写的前提
Git 友好	纯文本文件天然支持 diff、merge、history
足够	50 个小文件的 CRUD 不需要数据库
零依赖	不需要 SQLite、LevelDB 或任何外部存储

为什么 Vercel AI SDK

维度	分析
统一接口	一套代码支持 OpenAI / Anthropic / Google / 兼容 API
Tool calling	内置 Zod schema → JSON Schema 转换，自动处理多步 tool calling 循环
流式输出	`streamText` 提供开箱即用的 AsyncIterable
TypeScript 原生	类型安全，与项目技术栈一致

为什么无状态原子操作

CLI-first 设计：外部 Agent（Claude Code、Cursor、Cline）通过 shell 驱动 FlowCabal。每次命令独立完成：加载→执行→持久化→返回。无需维护长生命周期对象。

为什么不需要版本缓存

设计哲学："如果用户不重视生成的东西，那我们也认为它没有价值。"每次执行结果直接覆盖，Workspace 不维护多版本历史。用户自行决定是否保留（通过 git 或手动备份）。

CLI 命令

init

初始化项目，在当前目录创建 `.flowcabal/` 目录。

workspace

```
# 创建 workspace
flowcabal workspace create <name>

# 列出所有 workspace
flowcabal workspace list

# 切换当前 workspace
flowcabal workspace switch <id>

# 查看状态
flowcabal workspace status [id]

# 删除 workspace
flowcabal workspace delete <id>
```

llm

```
# 列出所有配置
flowcabal llm list

# 添加配置（交互式）
flowcabal llm add <name>

# 删除配置
flowcabal llm remove <name>

# 设为默认
flowcabal llm set-default <name>
```

node

```
# 创建节点（自动加入 target）
flowcabal node add <label>

# 删除节点
flowcabal node rm <id>

# 重命名节点
flowcabal node rename <id> <label>

# 列出所有节点
flowcabal node list

# 查看节点详情
flowcabal node cat <id>

# 插入 ref block（建立 DAG 连接）
flowcabal node ins-ref <id> <upstream>

# 插入 literal block（静态文本）
flowcabal node ins-literal <id> --content "文本内容"

# 插入 inject block（Agent 按 hint 注入内容）
flowcabal node ins-inject <id> --hint "注入提示"

# 删除 block
```

```
flowcabal node rm-block <id> <index>

# 将节点加入/移出执行目标
flowcabal node target <id>
flowcabal node untarget <id>
```

run

```
# 执行全部 target 节点
flowcabal run

# 预览执行顺序（不执行）
flowcabal run preview
```

memory

```
# 交互式对话
flowcabal memory chat

# 添加手稿
flowcabal memory add-manuscript <path>
```